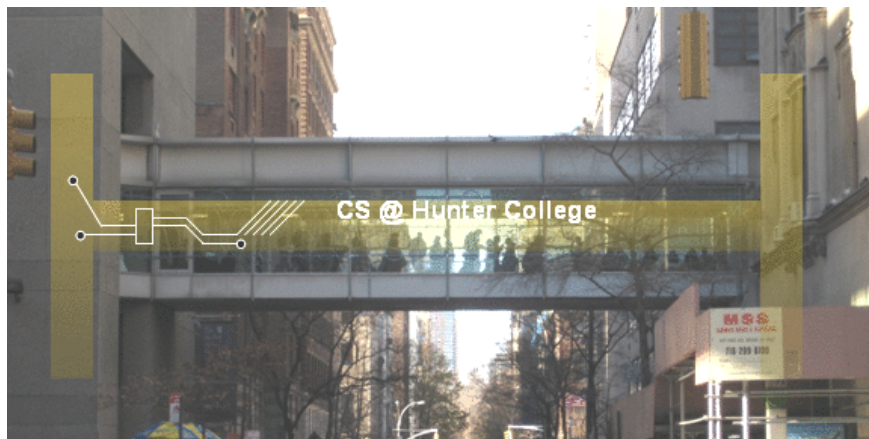


CSci 127: Introduction to Computer Science



hunter.cuny.edu/csci

Today's Topics



- Recap: Functions & Top Down Design
- Mapping Spatial Data
- Random Numbers
- Indefinite Loops

Today's Topics



- **Recap: Functions & Top Down Design**
- Mapping Spatial Data
- Random Numbers
- Indefinite Loops

Challenge:

```
def prob4(amy, beth):  
    if amy > 4:  
        print("Easy case")  
        kate = -1  
    else:  
        print("Complex case")  
        kate = helper(amy,beth)  
    return(kate)
```

```
def helper(meg,jo):  
    s = ""  
    for j in range(meg):  
        print(j, ": ", jo[j])  
        if j % 2 == 0:  
            s = s + jo[j]  
            print("Building s:", s)  
    return(s)
```

- What are the formal parameters for the functions?

- What is the output of:

```
r = prob4(4,"city")  
print("Return: ", r)
```

- What is the output of:

```
r = prob4(2,"university")  
print("Return: ", r)
```

Challenge:

```
def prob4(amy, beth):  
    if amy > 4:  
        print("Easy case")  
        kate = -1  
    else:  
        print("Complex case")  
        kate = helper(amy,beth)  
    return(kate)
```

```
def helper(meg,jo):  
    s = ""  
    for j in range(meg):  
        print(j, ": ", jo[j])  
        if j % 2 == 0:  
            s = s + jo[j]  
            print("Building s:", s)  
    return(s)
```

- What are the formal parameters for the functions?

Challenge:

```
def prob4(amy, beth):  
    if amy > 4:  
        print("Easy case")  
        kate = -1  
    else:  
        print("Complex case")  
        kate = helper(amy,beth)  
    return(kate)
```

**Formal
Parameters**

```
def helper(meg,jo):  
    s = ""  
    for j in range(meg):  
        print(j, ": ", jo[j])  
        if j % 2 == 0:  
            s = s + jo[j]  
        print("Building s:", s)  
    return(s)
```

- What are the formal parameters for the functions?

Challenge:

```
def prob4(amy, beth):  
    if amy > 4:  
        print("Easy case")  
        kate = -1  
    else:  
        print("Complex case")  
        kate = helper(amy,beth)  
    return(kate)
```

```
def helper(meg,jo):  
    s = ""  
    for j in range(meg):  
        print(j, ": ", jo[j])  
        if j % 2 == 0:  
            s = s + jo[j]  
            print("Building s:", s)  
    return(s)
```

- What is the output of:
 r = prob4(4,"city")
 print("Return: ", r)
- What is the output of:
 r = prob4(2,"university")
 print("Return: ", r)

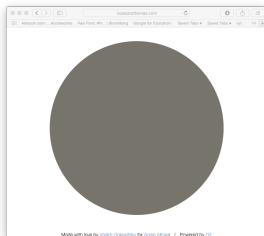
Python Tutor

```
def prob4(any, beth):  
    if any > 4:  
        print("Easy case")  
        kate = -1  
    else:  
        print("Complex case")  
        kate = helper(any, beth)  
    return(kate)
```

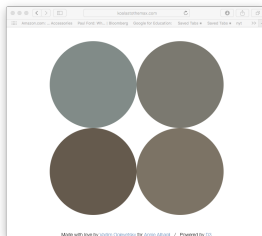
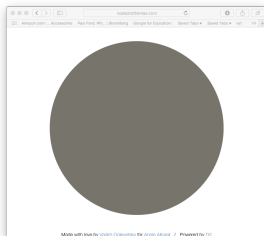
```
def helper(meg, jo):  
    s = ""  
    for j in range(meg):  
        print(j, ": ", jo[j])  
        if j % 2 == 0:  
            s = s + jo[j]  
        print("Building s:", s)  
    return(s)
```

(Demo with pythonTutor)

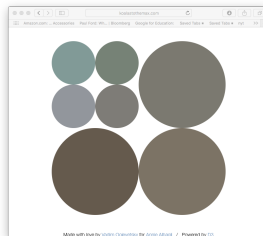
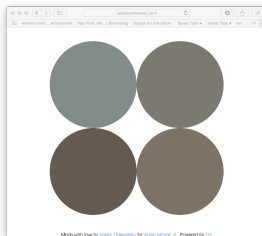
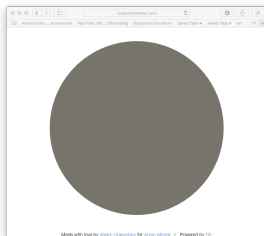
From Last Time: koalas



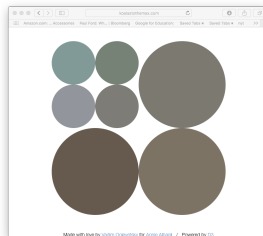
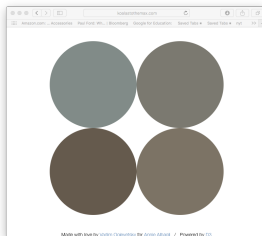
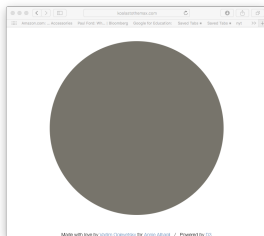
From Last Time: koalas



From Last Time: koalas

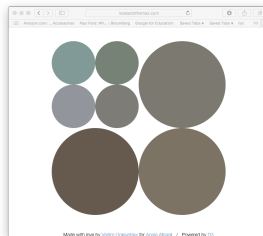
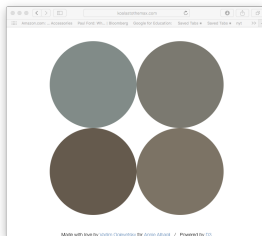
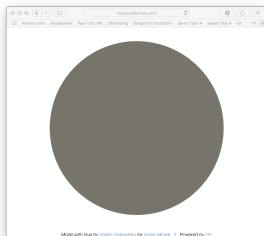


From Last Time: koalas

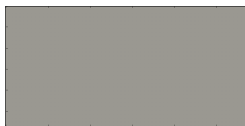


<http://koalastothemax.com>

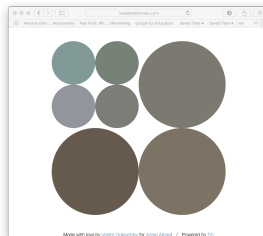
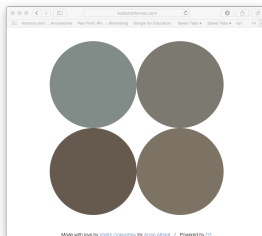
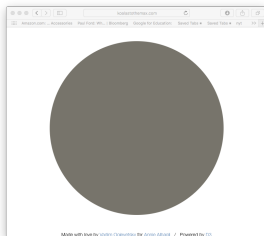
From Last Time: koalas



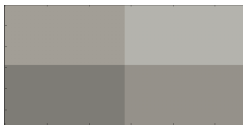
<http://koalastothemax.com>



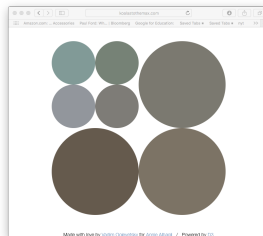
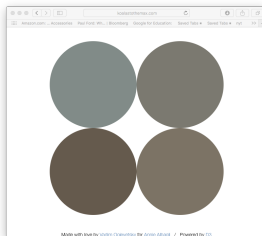
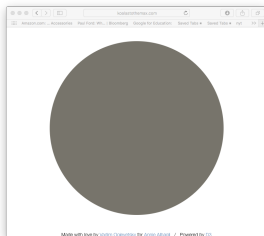
From Last Time: koalas



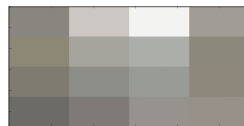
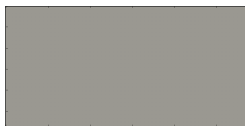
<http://koalastothemax.com>



From Last Time: koalas



<http://koalastothemax.com>



From Last Time: koalas

Process:



Get template
from github



Fill in missing
functions



Test locally
idle3/python3



Submit to
Gradescope

From Last Time: koalas



```
69 def main():
70     inFile = input('Enter image file name: ')
71     img = plt.imread(inFile)
72
73     #Divides the image in 1/2, 1/4, 1/8, ... 1/2^8, and displays each:
74     for i in range(8):
75         img2 = img.copy()    #Make a copy to average
76         quarter(img2,i)      #Split in half i times, and average regions
77
78         plt.imshow(img2)     #Load our new image into pyplot
79         plt.show()           #Show the image (waits until closed to continue)
80
81     #Shows the original image:
82     plt.imshow(img)          #Load image into pyplot
83     plt.show()               #Show the image (waits until closed to continue)
84
85
```

From Last Time: koalas



```
69 def main():
70     inFile = input('Enter image file name: ')
71     img = plt.imread(inFile)
72
73     #Divides the image in 1/2, 1/4, 1/8, ... 1/2^8, and displays each:
74     for i in range(8):
75         img2 = img.copy()    #Make a copy to average
76         quarter(img2,i)      #Split in half i times, and average regions
77
78         plt.imshow(img2)     #Load our new image into pyplot
79         plt.show()           #Show the image (waits until closed to continue)
80
81     #Shows the original image:
82     plt.imshow(img)          #Load image into pyplot
83     plt.show()               #Show the image (waits until closed to continue)
84
85
```

- The `main()` is written for you.

From Last Time: koalas



```
69 def main():
70     inFile = input('Enter image file name: ')
71     img = plt.imread(inFile)
72
73     #Divides the image in 1/2, 1/4, 1/8, ... 1/2^8, and displays each:
74     for i in range(8):
75         img2 = img.copy()    #Make a copy to average
76         quarter(img2,i)      #Split in half i times, and average regions
77
78         plt.imshow(img2)     #Load our new image into pyplot
79         plt.show()          #Show the image (waits until closed to continue)
80
81     #Shows the original image:
82     plt.imshow(img)         #Load image into pyplot
83     plt.show()              #Show the image (waits until closed to continue)
84
85
```

- The `main()` is written for you.
- Only fill in two functions: `average()` and `setRegion()`.

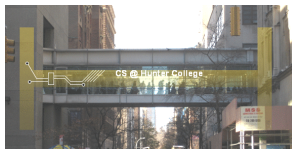
Top-Down Design

- The last example demonstrates **top-down design**: breaking into subproblems, and implementing each part separately.



Top-Down Design

- The last example demonstrates **top-down design**: breaking into subproblems, and implementing each part separately.
 - ▶ Break the problem into tasks for a “To Do” list.

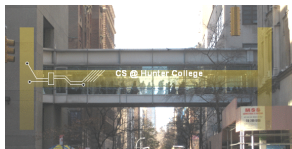


Top-Down Design

- The last example demonstrates **top-down design**: breaking into subproblems, and implementing each part separately.
 - ▶ Break the problem into tasks for a “To Do” list.
 - ▶ Translate list into function names & inputs/returns.



Top-Down Design



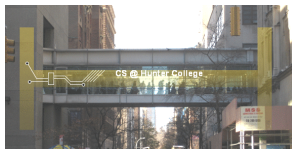
- The last example demonstrates **top-down design**: breaking into subproblems, and implementing each part separately.
 - ▶ Break the problem into tasks for a “To Do” list.
 - ▶ Translate list into function names & inputs/returns.
 - ▶ Implement the functions, one-by-one.

Top-Down Design



- The last example demonstrates **top-down design**: breaking into subproblems, and implementing each part separately.
 - ▶ Break the problem into tasks for a “To Do” list.
 - ▶ Translate list into function names & inputs/returns.
 - ▶ Implement the functions, one-by-one.
- Excellent approach since you can then test each part separately before adding it to a large program.

Top-Down Design



- The last example demonstrates **top-down design**: breaking into subproblems, and implementing each part separately.
 - ▶ Break the problem into tasks for a “To Do” list.
 - ▶ Translate list into function names & inputs/returns.
 - ▶ Implement the functions, one-by-one.
- Excellent approach since you can then test each part separately before adding it to a large program.
- Very common when working with a team: each has their own functions to implement and maintain.

Challenge:

- Write the missing functions for the program:

```
def main():  
    tess = setUp()      #Returns a purple turtle with pen up.  
    for i in range(5):  
        x,y = getInput()    #Asks user for two numbers.  
        markLocation(tess,x,y) #Move tess to (x,y) and stamp.
```

Challenge:

- Write the missing functions for the program:

```
def main():  
    tess = setUp()          #Returns a purple turtle with pen up.  
    for i in range(5):  
        x,y = getInput()      #Asks user for two numbers.  
        markLocation(tess,x,y) #Move tess to (x,y) and stamp.
```

Group Work: Fill in Missing Pieces

```
def main():  
    tess = setUp()      #Returns a purple turtle with pen up.  
    for i in range(5):  
        x,y = getInput()    #Asks user for two numbers.  
        markLocation(tess,x,y) #Move tess to (x,y) and stamp.
```

Group Work: Fill in Missing Pieces

- 1 Write import statements.

```
import turtle
```

```
def main():  
    tess = setUp()      #Returns a purple turtle with pen up.  
    for i in range(5):  
        x,y = getInput()    #Asks user for two numbers.  
        markLocation(tess,x,y) #Move tess to (x,y) and stamp.
```

Third Part: Fill in Missing Pieces

- 1 Write import statements.
- 2 Write down new function names and inputs.

```
import turtle
def setUp():
    #FILL IN
def getInput():
    #FILL IN
def markLocation(t,x,y):
    #FILL IN

def main():
    tess = setUp()      #Returns a purple turtle with pen up.
    for i in range(5):
        x,y = getInput()      #Asks user for two numbers.
        markLocation(tess,x,y) #Move tess to (x,y) and stamp.
```

Third Part: Fill in Missing Pieces

- 1 Write import statements.
- 2 Write down new function names and inputs.
- 3 Fill in return values.

```
import turtle

def setUp():
    #FILL IN
    return(newTurtle)

def getInput():
    #FILL IN
    return(x,y)

def markLocation(t,x,y):
    #FILL IN


def main():
    tess = setUp()      #Returns a purple turtle with pen up.
    for i in range(5):
        x,y = getInput()      #Asks user for two numbers.
        markLocation(tess,x,y) #Move tess to (x,y) and stamp.
```

Third Part: Fill in Missing Pieces

- 1 Write import statements.
- 2 Write down new function names and inputs.
- 3 Fill in return values.
- 4 Fill in body of functions.

```
import turtle

def setUp():
    newTurtle = turtle.Turtle()
    newTurtle.color("purple")
    newTurtle.penup()
    return(newTurtle)

def getInput():
    x = int(input('Enter x: '))
    y = int(input('Enter y: '))
    return(x,y)

def markLocation(t,x,y):
    t.goto(x,y)
    t.stamp()

def main():
    tess = setUp()      #Returns a purple turtle with pen up.
    for i in range(5):
        x,y = getInput()      #Asks user for two numbers.
```


Challenge:

- Write a function that takes a number as an input and prints its corresponding name.

Challenge:

- Write a function that takes a number as an input and prints its corresponding name.
- For example,

Challenge:

- Write a function that takes a number as an input and prints its corresponding name.
- For example,
 - ▶ `num2string(0)` returns: zero

Challenge:

- Write a function that takes a number as an input and prints its corresponding name.
- For example,
 - ▶ `num2string(0)` returns: zero
 - ▶ `num2string(1)` returns: one

Challenge:

- Write a function that takes a number as an input and prints its corresponding name.
- For example,
 - ▶ `num2string(0)` returns: zero
 - ▶ `num2string(1)` returns: one
 - ▶ `num2string(2)` returns: two

Challenge:

- Write a function that takes a number as an input and prints its corresponding name.
- For example,
 - ▶ `num2string(0)` returns: zero
 - ▶ `num2string(1)` returns: one
 - ▶ `num2string(2)` returns: two
- You may assume that only single digits, 0,1,...,9, are given as input.

Python Tutor



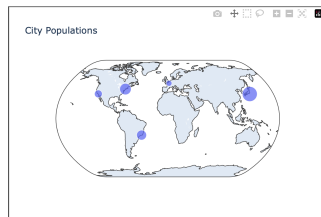
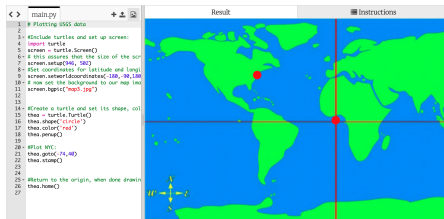
(numsConvert.py on On github)

Today's Topics



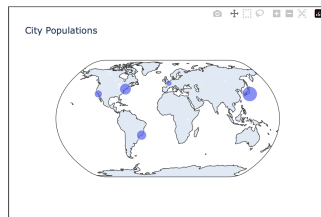
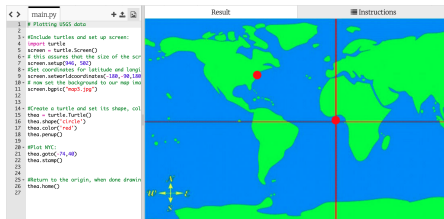
- Recap: Functions & Top Down Design
- **Mapping Spatial Data**
- Random Numbers
- Indefinite Loops

Mapping Spatial Data



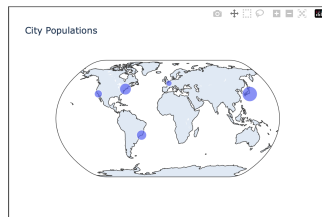
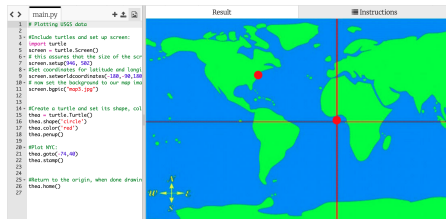
- Spatial or GIS data refers to information about a specific place.

Mapping Spatial Data



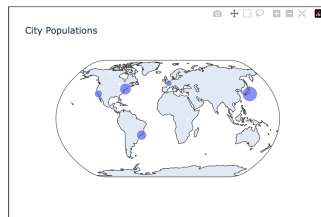
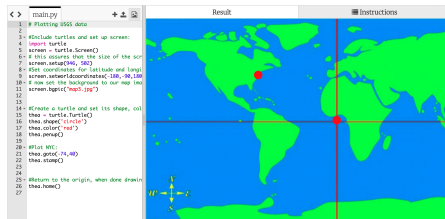
- Spatial or GIS data refers to information about a specific place.
- Geographic Information Systems (GIS) are the systems used to store, manipulate, and visualize geographic data.

Mapping Spatial Data



- Spatial or GIS data refers to information about a specific place.
- Geographic Information Systems (GIS) are the systems used to store, manipulate, and visualize geographic data.
- In Lab 9, we outline two approaches:
 - ▶ Using our beloved turtles to stamp locations on an underlying image.

Mapping Spatial Data



- Spatial or GIS data refers to information about a specific place.
- Geographic Information Systems (GIS) are the systems used to store, manipulate, and visualize geographic data.
- In Lab 9, we outline two approaches:
 - ▶ Using our beloved turtles to stamp locations on an underlying image.
 - ▶ Using Plotly Express to make interactive maps.

Plotly.py



- An open-source graphing library for Python.

Plotly.py



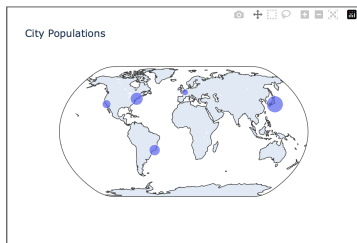
- An open-source graphing library for Python.
- Built on top of the JavaScript `plotly.js`.

Plotly.py

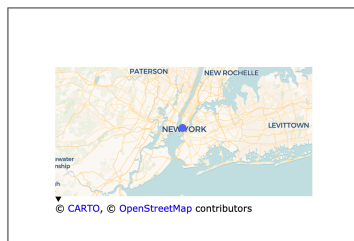


- An open-source graphing library for Python.
- Built on top of the JavaScript `plotly.js`.
- Can build stand-alone & browser-based charts, graphs, and maps.
- We will use the simplified interface: Plotly Express (`px`).

Some Plotly Maps



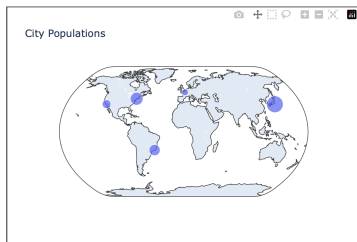
Outline Maps



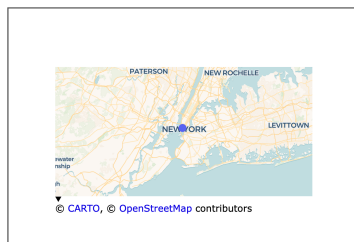
Tiled Maps

- Many different maps available in Plotly (see [Plotly map gallery](#)).
- Focus on Plotly Express (px) scatter plot maps on outline maps & tiled maps.

Some Plotly Maps



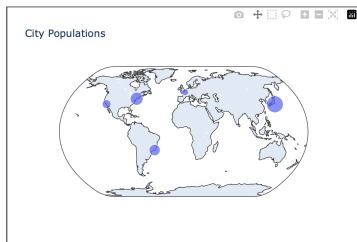
Outline Maps



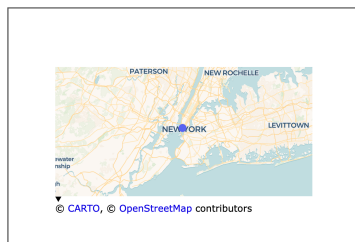
Tiled Maps

- Many different maps available in Plotly (see [Plotly map gallery](#)).
- Focus on Plotly Express (px) scatter plot maps on outline maps & tiled maps.
 - ▶ **Outline maps:** draws outline of regions. Use `px.scatter_geo()`

Some Plotly Maps



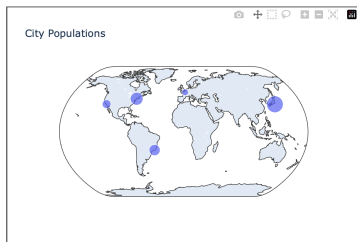
Outline Maps



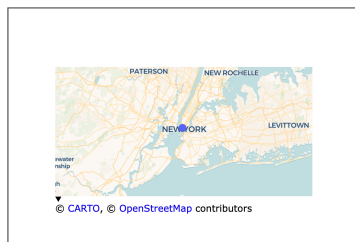
Tiled Maps

- Many different maps available in Plotly (see [Plotly map gallery](#)).
- Focus on Plotly Express (px) scatter plot maps on outline maps & tiled maps.
 - ▶ **Outline maps:** draws outline of regions. Use `px.scatter_geo()`
 - ▶ **Tiled maps:** background details changes with zoom. Use `px.scatter_map()`.

Some Plotly Maps



Outline Maps



Tiled Maps

- Many different maps available in Plotly (see [Plotly map gallery](#)).
- Focus on Plotly Express (px) scatter plot maps on outline maps & tiled maps.
 - ▶ **Outline maps:** draws outline of regions. Use `px.scatter_geo()`
 - ▶ **Tiled maps:** background details changes with zoom. Use `px.scatter_map()`.
 - ▶ **Graph Objects:** part of base Plotly, Can use to add markers– more control, but more work. Use `go.Figure(go.Scattermap())`.

Plotly

- To import libraries for Plotly Express, Graph objects, & panda:

python import plotly.express as px For Plotly Express import
plotly.graph_objects as go To use markers import pandas as pd To use dataframes

```
import plotly.express as px          #For Plotly Express
import plotly.graph_objects as go     #To use markers
import pandas as pd                  #To use dataframes
```

Plotly

- To import libraries for Plotly Express, Graph objects, & panda:
python import plotly.express as px For Plotly Express import
plotly.graph_objects as go To use markers import pandas as pd To use dataframes

```
import plotly.express as px          #For Plotly Express
import plotly.graph_objects as go    #To use markers
import pandas as pd                 #To use dataframes
```

- Create a tiled map:

```
import plotly.express as px
import pandas as pd
fig = px.scatter_map(cuny_df, lat="Latitude", lon="Longitude",
                    hover_name="Campus",
                    title="CUNY Campuses",
                    zoom=9)
fig.show()
fig.write_html('cunyLocations.html')
```

Plotly

- To import libraries for Plotly Express, Graph objects, & panda:
python import plotly.express as px For Plotly Express import
plotly.graph_objects as go To use markers import pandas as pd To use dataframes

```
import plotly.express as px          #For Plotly Express
import plotly.graph_objects as go    #To use markers
import pandas as pd                  #To use dataframes
```

- Create a tiled map:

```
import plotly.express as px
import pandas as pd
fig = px.scatter_map(cuny_df, lat="Latitude", lon="Longitude",
                    hover_name="Campus",
                    title="CUNY Campuses",
                    zoom=9)
fig.show()
fig.write_html('cunyLocations.html')
```

- Make markers by hand:

```
fig = go.Figure(go.Scattermap(lat=[40.7128], lon=[-74.0060],
                             mode='markers',
                             marker=go.scattermap.Marker(size=14),
                             text=['New York City'])))
```

Plotly

- To import libraries for Plotly Express, Graph objects, & panda:
python import plotly.express as px For Plotly Express import
plotly.graph_objects as go To use markers import pandas as pd To use dataframes

```
import plotly.express as px          #For Plotly Express
import plotly.graph_objects as go    #To use markers
import pandas as pd                  #To use dataframes
```

- Create a tiled map:

```
import plotly.express as px
import pandas as pd
fig = px.scatter_map(cuny_df, lat="Latitude", lon="Longitude",
                    hover_name="Campus",
                    title="CUNY Campuses",
                    zoom=9)
fig.show()
fig.write_html('cunyLocations.html')
```

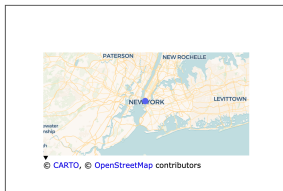
- Make markers by hand:

```
fig = go.Figure(go.Scattermap(lat=[40.7128], lon=[-74.0060],
                             mode='markers',
                             marker=go.scattermap.Marker(size=14),
                             text=['New York City'])))
```

- Save map:

```
fig.write_html('first_map.html')
```

Marker Example



```
import plotly.graph_objects as go  #Including for markers

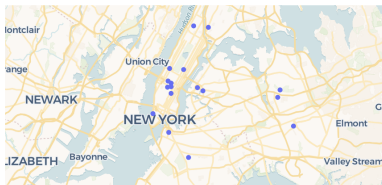
#Add a marker for NYC:
fig = go.Figure(go.Scattermap(lat=[40.7128], lon=[-74.0060],
                             mode='markers',
                             marker=go.scattermap.Marker(size=14),
                             text=['New York City']))

#Zoom and center the map on NYC:
fig.update_layout(hovermode='closest',
                  map=dict(center=go.layout.map.Center(lat=40.7,lon=-74),
                           zoom=8))

fig.show()
fig.write_html('first_map.html')
```


Tile Map Example

CUNY Campus



© CARTO, © OpenStreetMap contributors

```
import plotly.express as px
import pandas as pd
fig = px.scatter_map(cuny_df, lat="Latitude", lon="Longitude",
                    hover_name="Campus",
                    title="CUNY Campuses",
                    zoom=9)
fig.show()
fig.write_html('cunyLocations.html')
```

Today's Topics



- Recap: Functions & Top Down Design
- Mapping Spatial Data
- **Random Numbers**
- Indefinite Loops

Python's random package

- Python has a built-in package for generating pseudo-random numbers.

```
import turtle
import random

trex = turtle.Turtle()
trex.speed(10)

for i in range(100):
    trex.forward(10)
    a = random.randrange(0,360,90)
    trex.right(a)
```

Python's random package

- Python has a built-in package for generating pseudo-random numbers.
- To use:

```
import random
```

```
import turtle
import random

trey = turtle.Turtle()
trey.speed(10)

for i in range(100):
    trey.forward(10)
    a = random.randrange(0,360,90)
    trey.right(a)
```

Python's random package

- Python has a built-in package for generating pseudo-random numbers.

- To use:

```
import random
```

- Useful command to generate whole numbers:

```
random.randrange(start, stop, step)
```

which gives a number chosen randomly from the specified range.

```
import turtle
import random

trey = turtle.Turtle()
trey.speed(10)

for i in range(100):
    trey.forward(10)
    a = random.randrange(0, 360, 90)
    trey.right(a)
```

Python's random package

- Python has a built-in package for generating pseudo-random numbers.

- To use:

```
import random
```

- Useful command to generate whole numbers:

```
random.randrange(start, stop, step)
```

which gives a number chosen randomly from the specified range.

- Useful command to generate real numbers:

```
import turtle
import random

trex = turtle.Turtle()
trex.speed(10)

for i in range(100):
    trex.forward(10)
    a = random.randrange(0, 360, 90)
    trex.right(a)
```

Python's random package

- Python has a built-in package for generating pseudo-random numbers.

- To use:

```
import random
```

- Useful command to generate whole numbers:

```
random.randrange(start, stop, step)
```

which gives a number chosen randomly from the specified range.

- Useful command to generate real numbers:

```
random.random()
```

which gives a number chosen (uniformly) at random from $[0.0, 1.0)$.

```
import turtle
import random

trey = turtle.Turtle()
trey.speed(10)

for i in range(100):
    trey.forward(10)
    a = random.randrange(0, 360, 90)
    trey.right(a)
```

Python's random package

- Python has a built-in package for generating pseudo-random numbers.

- To use:

```
import random
```

- Useful command to generate whole numbers:

```
random.randrange(start, stop, step)
```

which gives a number chosen randomly from the specified range.

- Useful command to generate real numbers:

```
random.random()
```

which gives a number chosen (uniformly) at random from $[0.0, 1.0)$.

- Very useful for simulations, games, and testing.

```
import turtle
import random

trey = turtle.Turtle()
trey.speed(10)

for i in range(100):
    trey.forward(10)
    a = random.randrange(0, 360, 90)
    trey.right(a)
```


Trinket

```
import turtle
import random

trex = turtle.Turtle()
trex.speed(10)

for i in range(100):
    trex.forward(10)
    a = random.randrange(0,360,90)
    trex.right(a)
```

(Demo turtle
random walk)

Today's Topics



- Recap: Functions & Top Down Design
- Mapping Spatial Data
- Random Numbers
- **Indefinite Loops**

Challenge:

Predict what the code will do:

```
dist = int(input('Enter distance: '))
while dist < 0:
    print('Distances cannot be negative.')
    dist = int(input('Enter distance: '))
print('The distance entered is', dist)
```

Python Tutor

```
dist = int(input('Enter distance: '))
while dist < 0:
    print('Distances cannot be negative.')
    dist = int(input('Enter distance: '))
print('The distance entered is', dist)
```

(Demo with pythonTutor)

Indefinite Loops

- Indefinite loops repeat as long as the condition is true.

```
dist = int(input('Enter distance: '))
while dist < 0:
    print('Distances cannot be negative.')
    dist = int(input('Enter distance: '))
print('The distance entered is', dist)
```

```
#Spring 2012 Final Exam, #8
nums = [1,4,0,6,5,2,9,8,12]
print(nums)
i=0
while i < len(nums)-1:
    if nums[i] < nums[i+1]:
        nums[i], nums[i+1] = nums[i+1], nums[i]
    i=i+1
print(nums)
```

Indefinite Loops

```
dist = int(input('Enter distance: '))
while dist < 0:
    print('Distances cannot be negative.')
    dist = int(input('Enter distance: '))
print('The distance entered is', dist)
```

```
#Spring 2012 Final Exam, #8
nums = [1,4,0,6,5,2,9,8,12]
print(nums)
i=0
while i < len(nums)-1:
    if nums[i] < nums[i+1]:
        nums[i], nums[i+1] = nums[i+1], nums[i]
    i=i+1
print(nums)
```

- Indefinite loops repeat as long as the condition is true.
- Could execute the body of the loop zero times, 10 times, infinite number of times.

Indefinite Loops

```
dist = int(input('Enter distance: '))
while dist < 0:
    print('Distances cannot be negative.')
    dist = int(input('Enter distance: '))
print('The distance entered is', dist)
```

```
#Spring 2012 Final Exam, #8
nums = [1,4,0,6,5,2,9,8,12]
print(nums)
i=0
while i < len(nums)-1:
    if nums[i] < nums[i+1]:
        nums[i], nums[i+1] = nums[i+1], nums[i]
        i=i+1
print(nums)
```

- Indefinite loops repeat as long as the condition is true.
- Could execute the body of the loop zero times, 10 times, infinite number of times.
- The condition determines how many times.

Indefinite Loops

```
dist = int(input('Enter distance: '))
while dist < 0:
    print('Distances cannot be negative.')
    dist = int(input('Enter distance: '))
print('The distance entered is', dist)
```

```
#Spring 2012 Final Exam, #8
nums = [1,4,0,6,5,2,9,8,12]
print(nums)
i=0
while i < len(nums)-1:
    if nums[i] < nums[i+1]:
        nums[i], nums[i+1] = nums[i+1], nums[i]
    i=i+1
print(nums)
```

- Indefinite loops repeat as long as the condition is true.
- Could execute the body of the loop zero times, 10 times, infinite number of times.
- The condition determines how many times.
- Very useful for checking input, simulations, and games.

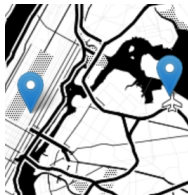
Indefinite Loops

```
dist = int(input('Enter distance: '))
while dist < 0:
    print('Distances cannot be negative.')
    dist = int(input('Enter distance: '))
print('The distance entered is', dist)
```

```
#Spring 2012 Final Exam, #8
nums = [1,4,0,6,5,2,9,8,12]
print(nums)
i=0
while i < len(nums)-1:
    if nums[i] < nums[i+1]:
        nums[i], nums[i+1] = nums[i+1], nums[i]
    i=i+1
print(nums)
```

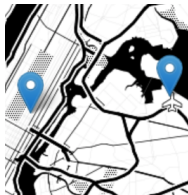
- Indefinite loops repeat as long as the condition is true.
- Could execute the body of the loop zero times, 10 times, infinite number of times.
- The condition determines how many times.
- Very useful for checking input, simulations, and games.
- More details next lecture...

Recap



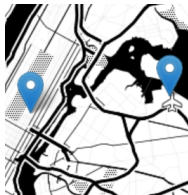
- Top-down design: breaking into subproblems, and implementing each part separately.

Recap



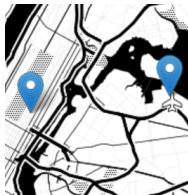
- Top-down design: breaking into subproblems, and implementing each part separately.
- Excellent approach: can then test each part separately before adding it to a large program.

Recap



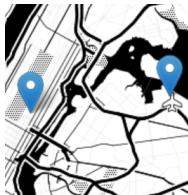
- Top-down design: breaking into subproblems, and implementing each part separately.
- Excellent approach: can then test each part separately before adding it to a large program.
- When possible, design so that your code is flexible to be reused (“code reuse”).

Recap



- Top-down design: breaking into subproblems, and implementing each part separately.
- Excellent approach: can then test each part separately before adding it to a large program.
- When possible, design so that your code is flexible to be reused (“code reuse”).
- Introduced a Python library, `Plotly` for creating interactive HTML maps.

Recap



- Top-down design: breaking into subproblems, and implementing each part separately.
- Excellent approach: can then test each part separately before adding it to a large program.
- When possible, design so that your code is flexible to be reused (“code reuse”).
- Introduced a Python library, `Plotly` for creating interactive HTML maps.
- Introduced `while` loops for repeating commands for an indefinite number of times.